UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Title

Submitted by:

Team Members

NAME	SRN
Arkul Prathamesh Shenoy	PES1UG23CS103
Arjun MS	PES1UG23CS102
Aryan Arun Hiremath	PES1UG23CS109
Atharw Rawal	PES1UG23CS119

Semester - Section

Faculty Name

January - May 2026

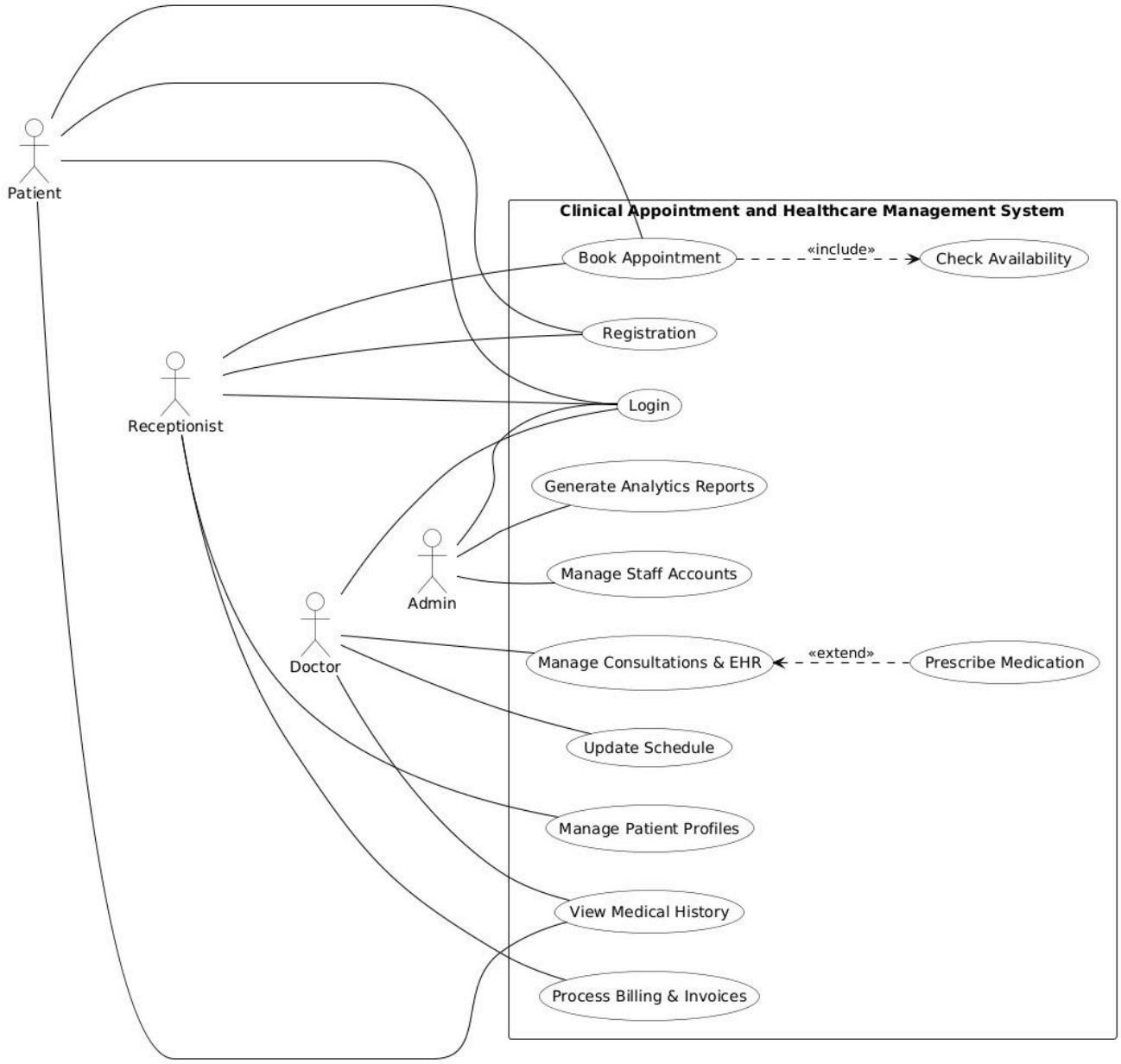
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

Problem Statement: Clinical Appointment and Healthcare Management System

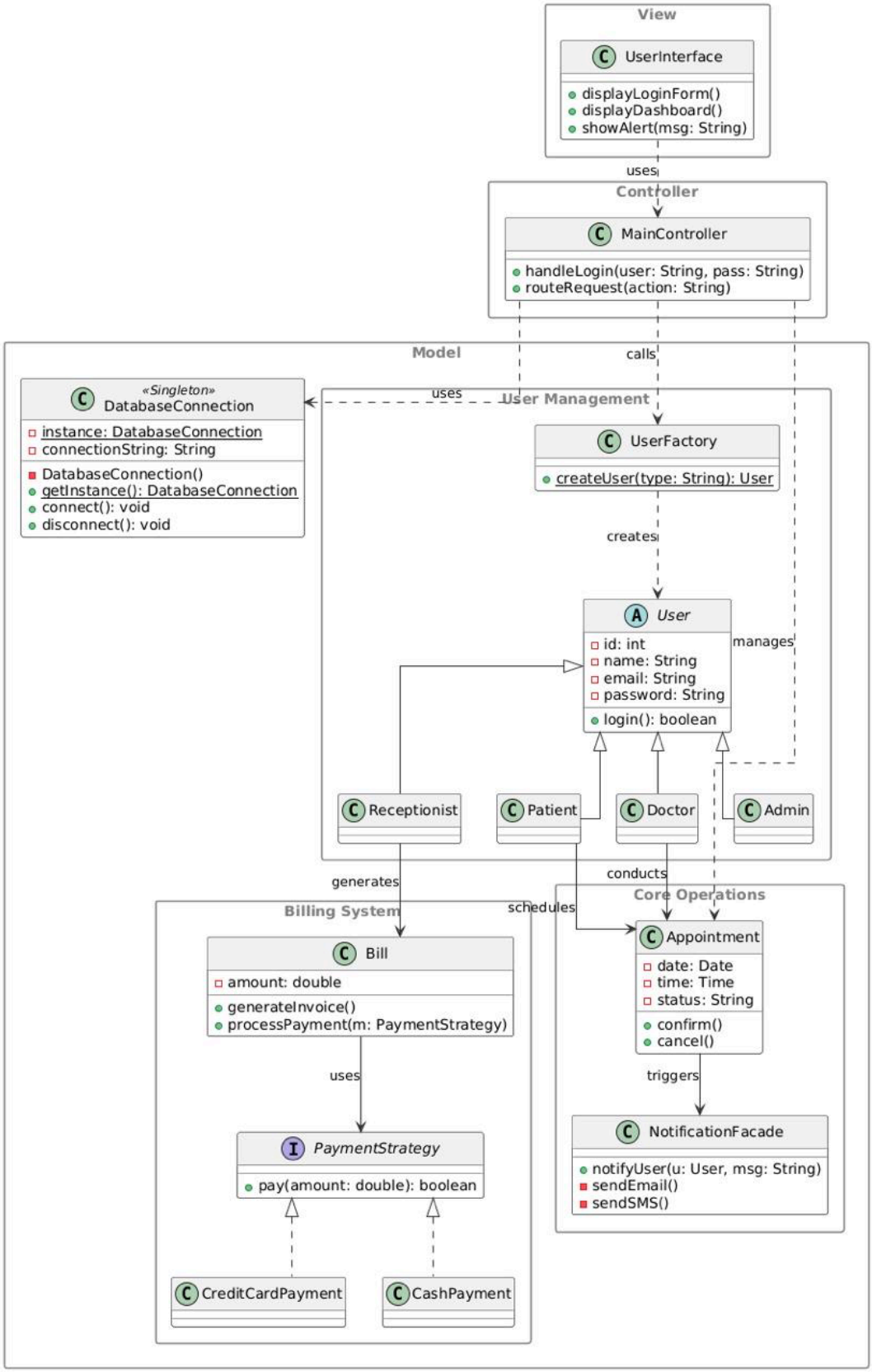
Key Features:

Models:

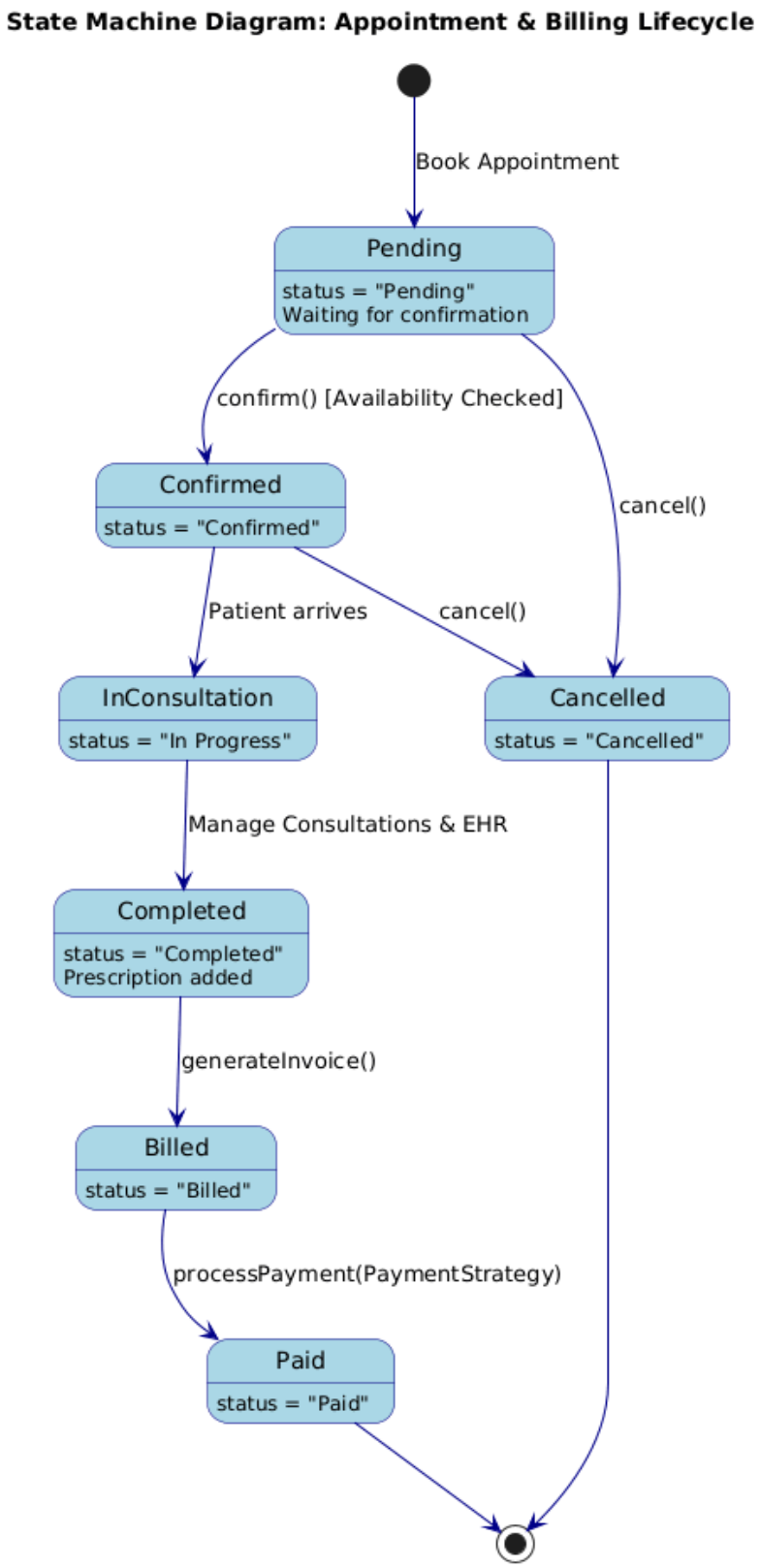
Use Case Diagram:



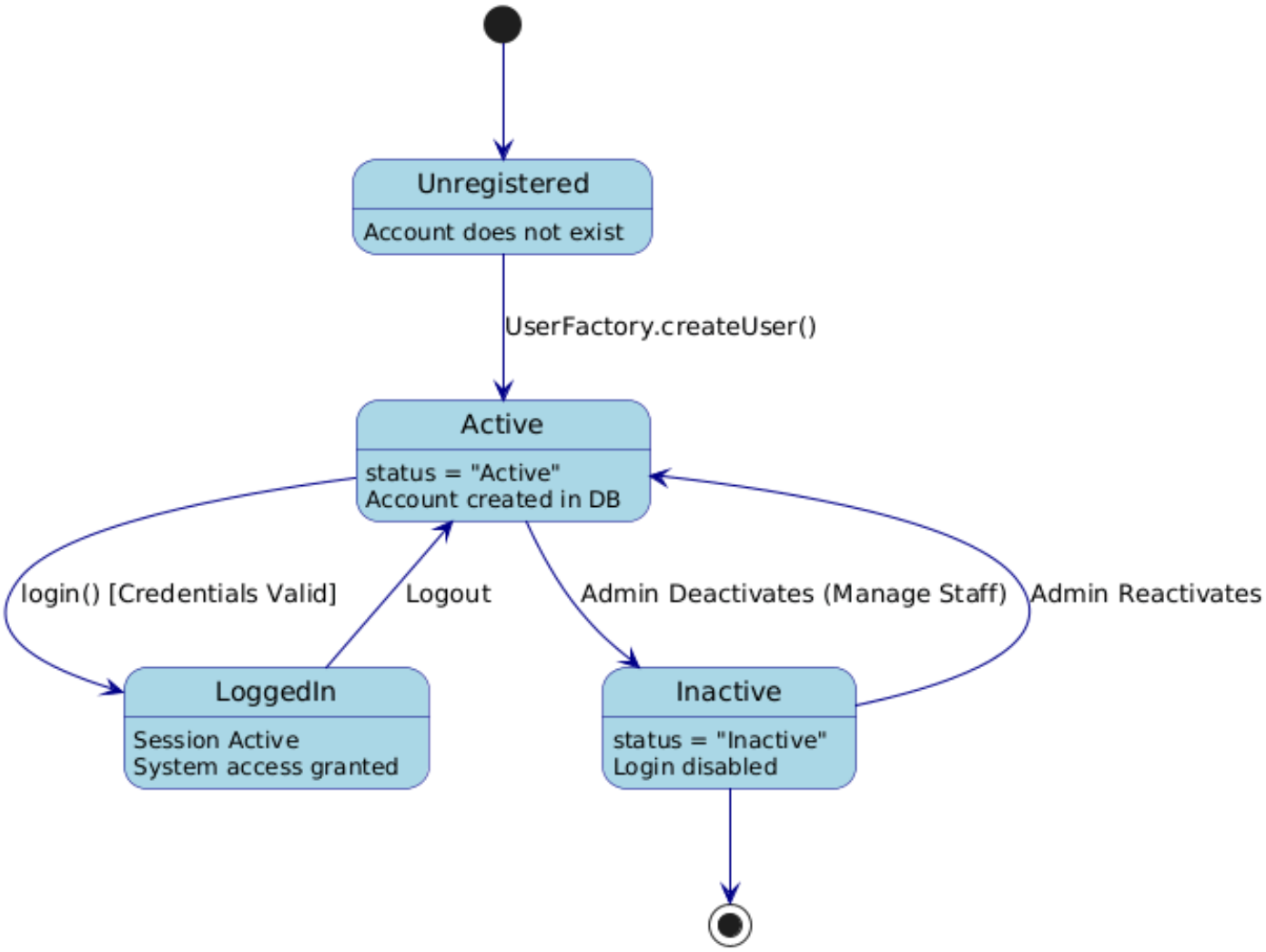
Class Diagram:



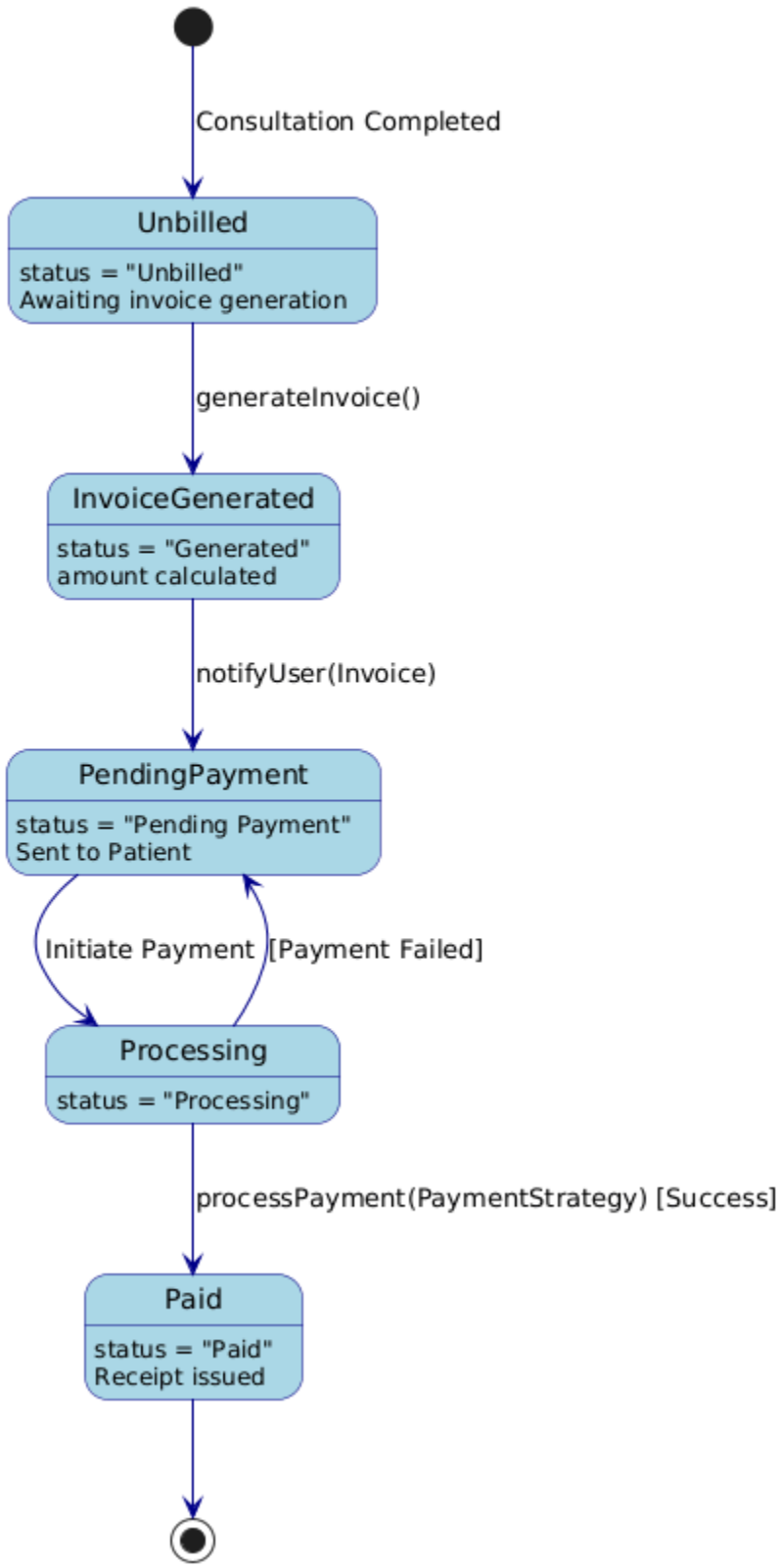
State Diagram:



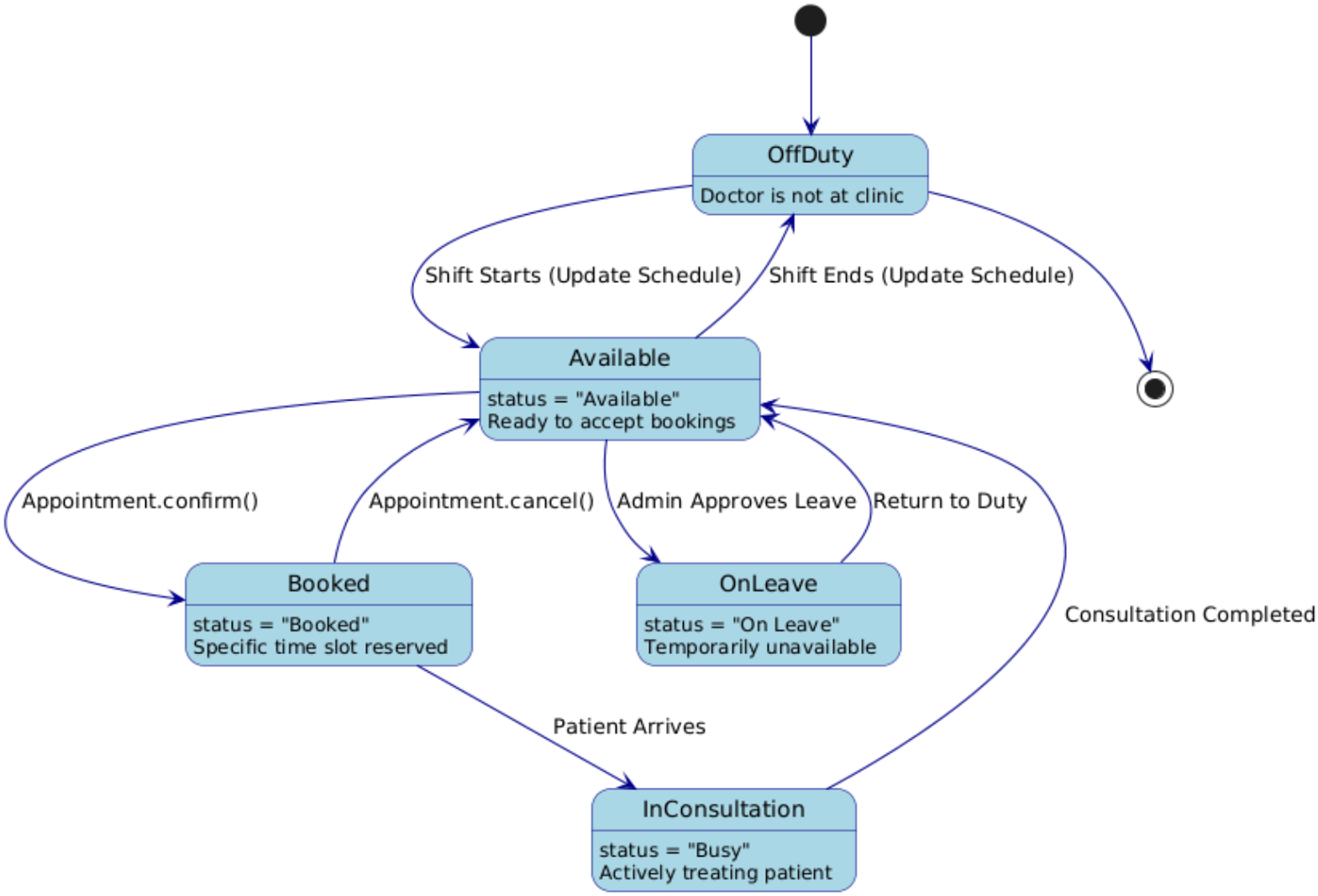
State Machine Diagram: User Account Lifecycle



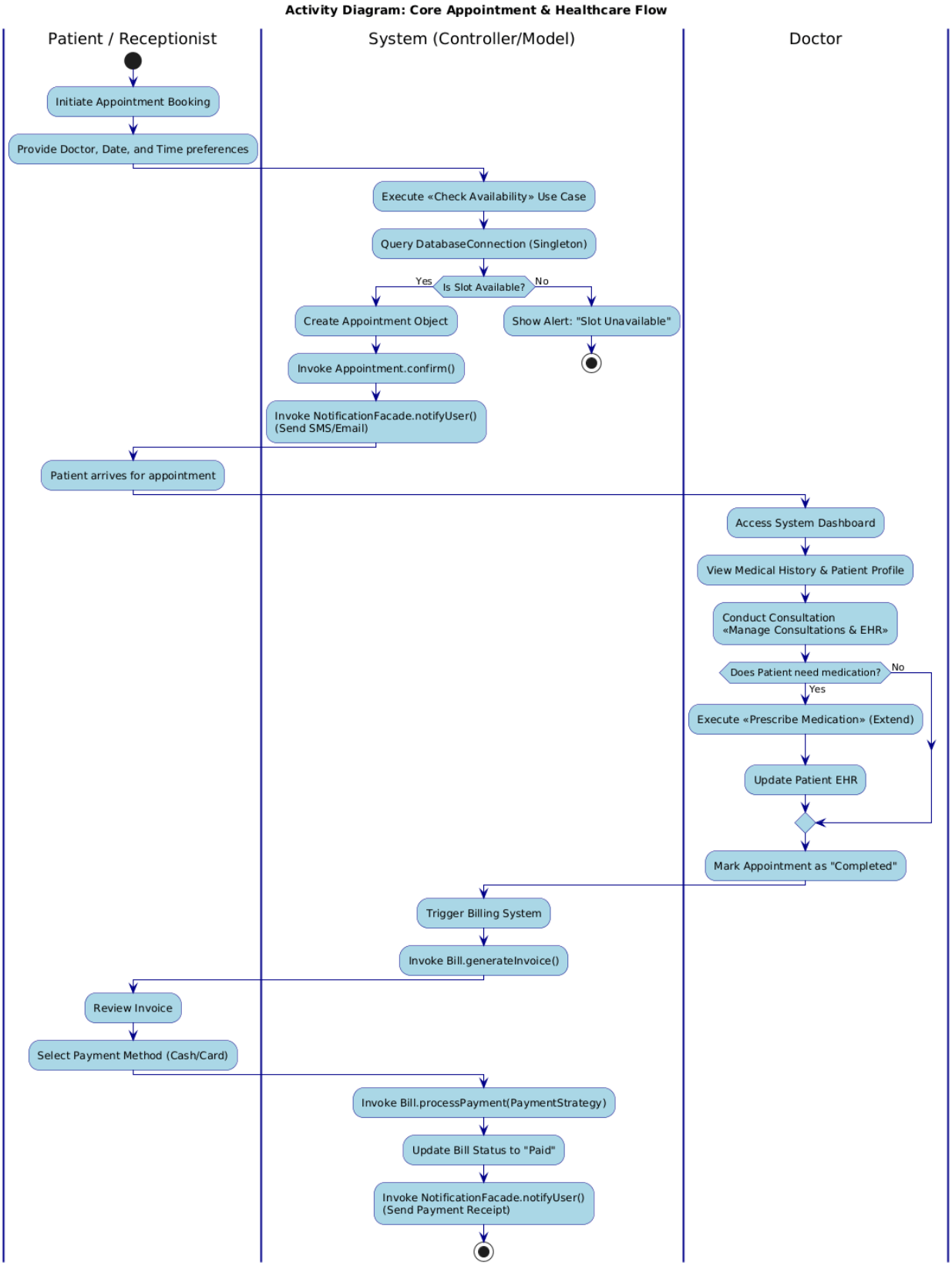
State Machine Diagram: Billing & Invoice Lifecycle



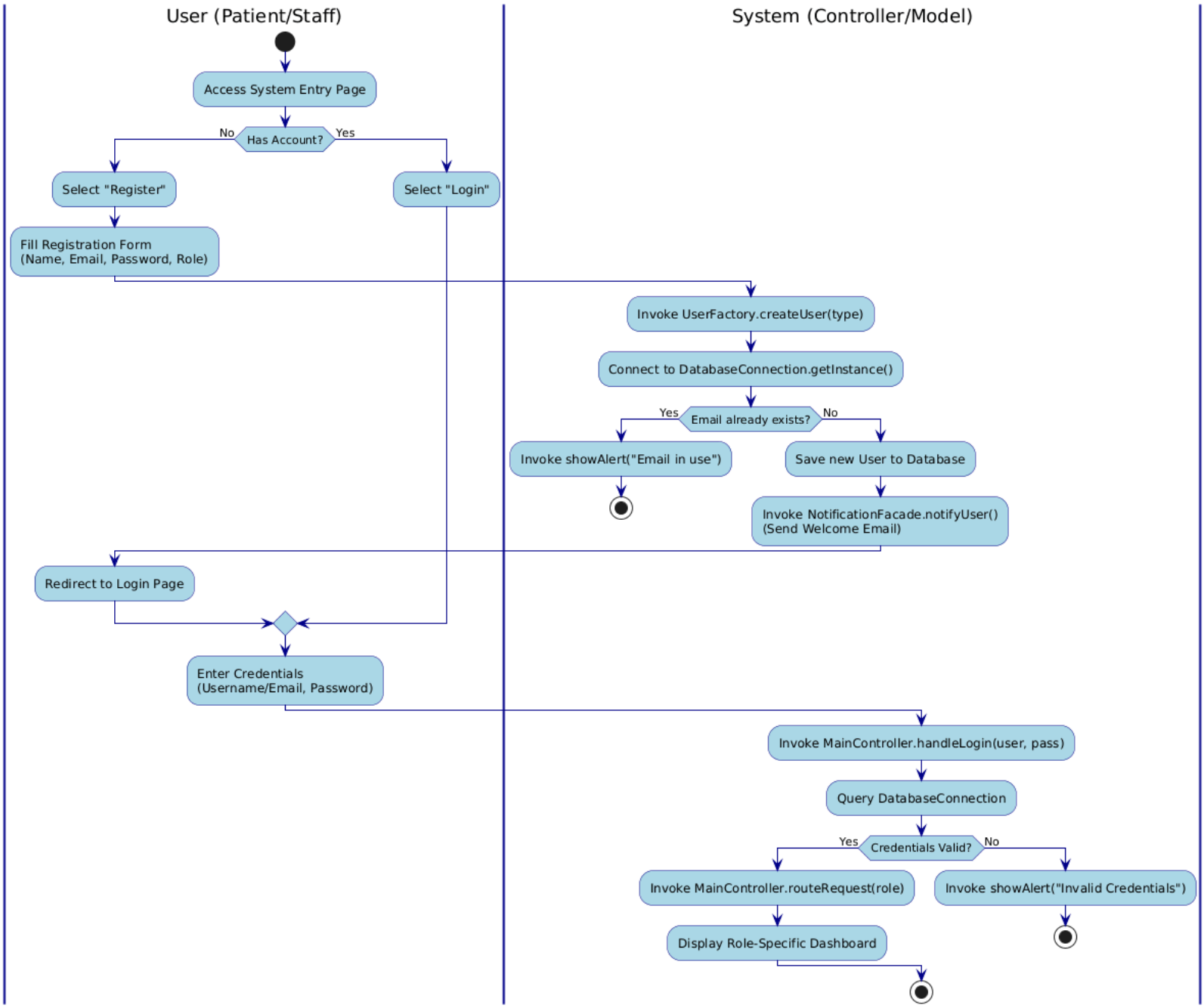
State Machine Diagram: Doctor Availability & Schedule



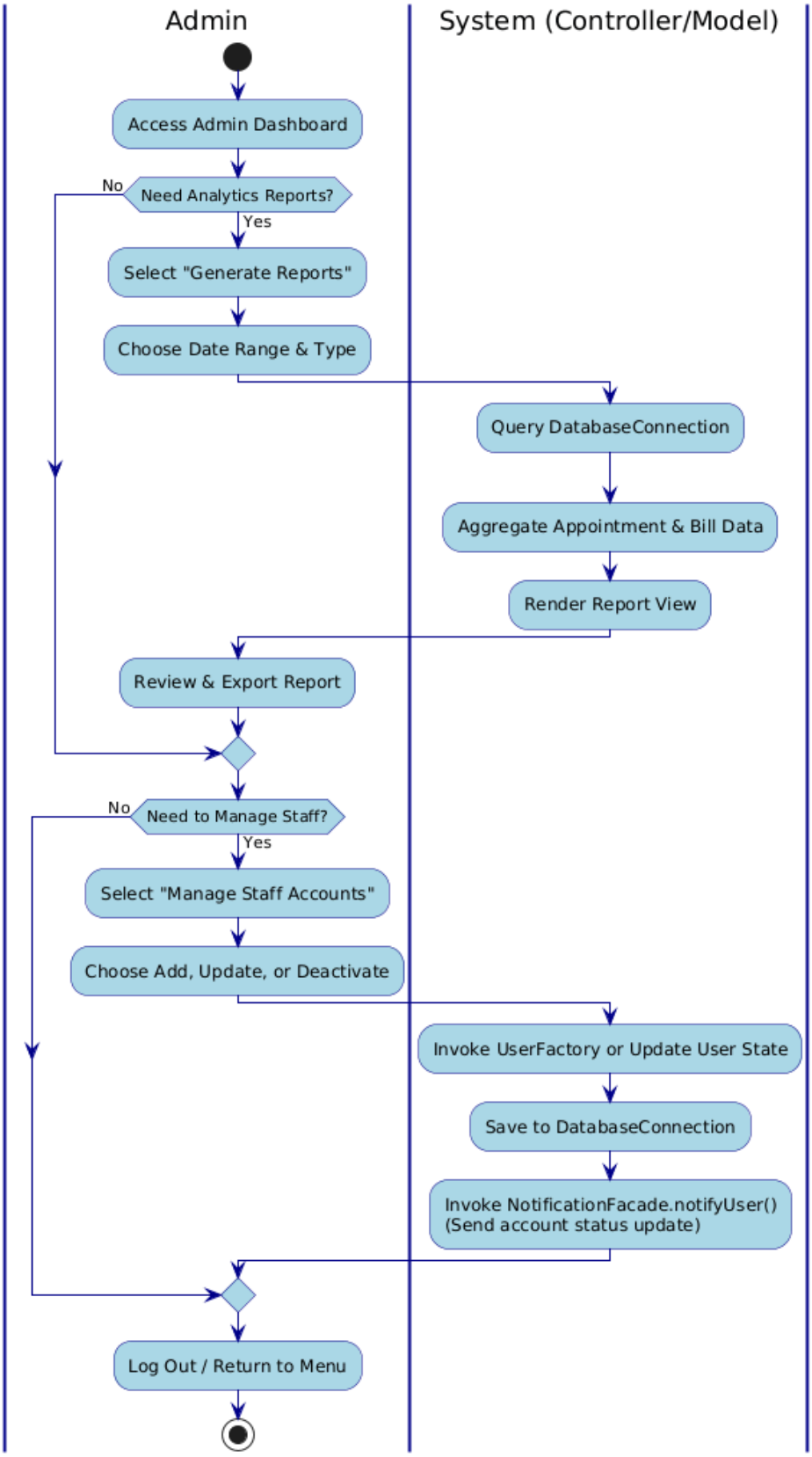
Activity Diagrams:



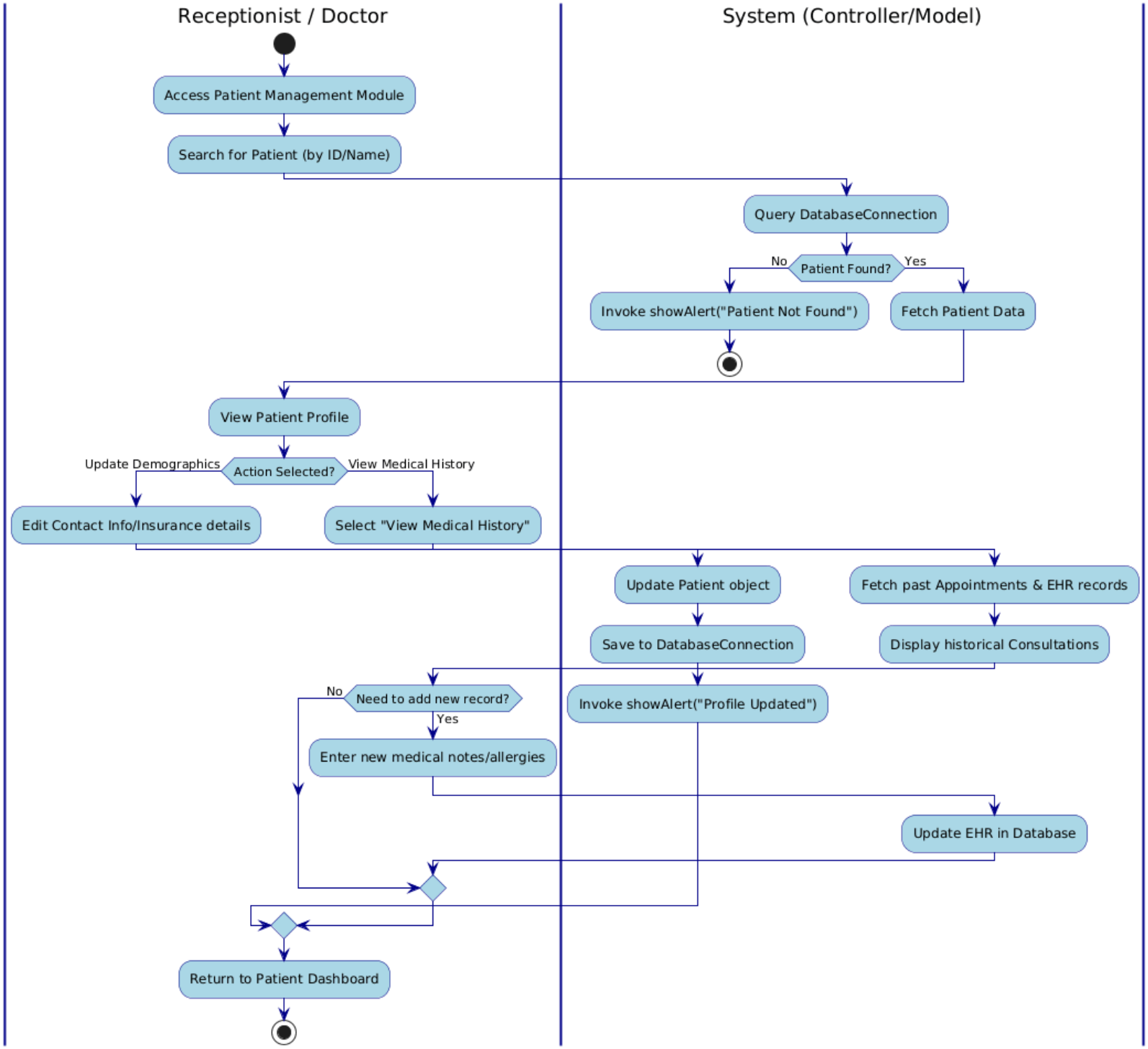
Activity Diagram: Registration & Login Flow



Activity Diagram: Admin Analytics & Staff Management



Activity Diagram: Patient Profile & Medical History Management



Design Principles, and Design Patterns:

Design Principles, and Design Patterns:

MVC Architecture used? Yes

Design Principles

- **Single Responsibility Principle (SRP):** Classes are designed to handle one specific functional area. For example, the `Bill` class is strictly responsible for invoice calculations and payment tracking, while the `Appointment` class solely manages scheduling data and status updates.
- **Open/Closed Principle (OCP):** The payment module is open for extension but closed for modification. By relying on the `PaymentStrategy` interface, the system can support new payment types (such as UPI or Insurance) in the future without requiring changes to the core `Bill` class.
- **Dependency Inversion Principle (DIP):** High-level business logic modules do not depend on low-level utility classes. Instead of the `Bill` class depending directly on concrete implementations like `CreditCardPayment`, it depends entirely on the `PaymentStrategy` abstraction.
- **Encapsulate What Varies:** The process of instantiating different user roles requires conditional logic that varies based on input. This creation logic is encapsulated entirely within the `UserFactory` rather than being scattered across the `MainController`.

Design Patterns

- **Singleton Pattern:** Applied to the `DatabaseConnection` class. This ensures that only one instance of the database connection exists throughout the application lifecycle, which manages resources efficiently and prevents concurrent connection leaks.
- **Factory Pattern:** Applied via the `UserFactory` class. It handles the creation of various user entity objects (Patient, Doctor, Receptionist, Admin). This centralizes object creation logic and keeps the main application controllers clean of direct instantiation code.
- **Strategy Pattern:** Applied using the `PaymentStrategy` interface and its concrete implementations (`CreditCardPayment`, `CashPayment`). This allows the `Bill` class to process payments interchangeably and dynamically switch between different payment algorithms at runtime.
- **Facade Pattern:** Applied to the `NotificationFacade`. It provides a unified, simplified interface for triggering alerts. It abstracts the underlying complexities of the individual SMS and Email subsystems, allowing the core operations to send notifications with a single method call.

Github link to the Codebase: (repository should be public)

https://github.com/

Screenshots

UI:

Individual contributions of the team members:

Name	**Module worked on**